

Análisis de la complejidad de operaciones en estructuras de listas

Belcar Santiago Cuentas-Zavala Infante

1. Código fuente

Se implementaron estructuras de datos abstractas con representaciones concretas distintas, según los requisitos de la tarea. El código fuente está disponible en https://github.com/dontwanttothink/basic_lists.

2. Hipótesis

Se desarrollaron hipótesis sobre la complejidad de tiempo de las operaciones implementadas. En esta sección, tómese $n \in \{0, 1, 2, \dots\}$ como la cantidad de elementos en la lista al momento de la operación.

2.1. Listas enlazadas

Se implementó una estructura lineal basada en listas enlazadas con los métodos descritos en el documento.

Importante

Nótese que los métodos `erase`, `addBefore` y `addAfter` se consideraron como respectivamente tomando el nodo a eliminar, el nodo antes del cual agregar un elemento, y el nodo después del cual agregar un elemento. No se toma el valor en sí, lo que permite en ocasiones omitir una búsqueda lineal. (Véase la implementación.)

2.1.1. Lista simplemente enlazada, sin cola

pushFront Se anticipa $\Theta(1)$ en cualquier caso, porque el flujo del algoritmo es idéntico para cualquier $n > 0$ y la cantidad de trabajo realizada es constante.

Operación	Peor caso	Caso promedio	Mejor caso
pushFront	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
pushBack	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
popFront	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
popBack	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
find	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
erase	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
addBefore	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
addAfter	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
isEmpty	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Cuadro 1: Resumen de las complejidades de tiempo para la lista simplemente enlazada, sin cola

pushBack Se anticipa $\Theta(n)$ en cualquier caso, porque para cualquier $n > 0$ se ejecuta una cantidad de trabajo constante por cada elemento de la lista, más una pequeña cantidad de trabajo que es constante e independiente de n .

popFront Se anticipa $\Theta(1)$ en cualquier caso, por razones idénticas a las dadas para pushFront.

popBack Se anticipa $\Theta(n)$ en cualquier caso, porque para cualquier $n > 0$ se necesita realizar una cantidad constante de trabajo por cada elemento anterior al último, más una cantidad pequeña de tiempo constante e independiente de n .

find Se anticipa $\Theta(n)$ en el peor caso, porque el flujo más costoso ocurre cuando el elemento está al final o no existe, y en aquel caso se realiza una cantidad de trabajo constante por cada elemento de la lista, más una cantidad de trabajo constante independiente de n .

Se anticipa $\Theta(1)$ en el mejor caso, porque el flujo menos costoso ocurre cuando el elemento es el primero, y en aquel caso se realiza solo una cantidad de trabajo constante.

Dado un elemento aleatorio de la lista, se anticipa $\Theta(n)$ en el caso promedio, porque la posición esperada del elemento es $\frac{n+1}{2}$, y se realiza una cantidad de trabajo constante por cada elemento anterior al elemento que se busca, más una cantidad de trabajo constante e independiente de n .

erase Se anticipa $\Theta(n)$ en el peor caso, $\Theta(1)$ en el mejor caso, y $\Theta(n)$ en el caso promedio, por razones idénticas a las dadas para find, ya que se necesita encontrar el antecesor del nodo a eliminar.

addBefore Se anticipa $\Theta(n)$ en el peor caso, $\Theta(1)$ en el mejor caso, y $\Theta(n)$ en el caso promedio, por razones idénticas a las dadas para find.

addAfter Se anticipa $\Theta(1)$ en cualquier caso, porque para todo $n > 0$ se realiza una cantidad constante de trabajo para cualquier $n > 0$.

isEmpty Se anticipa $\Theta(1)$ en cualquier caso, porque la implementación consiste en una verificación simple de igualdad que requiere una cantidad constante de trabajo.

2.1.2. Lista simplemente enlazada, con cola

Operación	Peor caso	Caso promedio	Mejor caso
pushFront	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
pushBack	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
popFront	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
popBack	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
find	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
erase	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
addBefore	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
addAfter	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
isEmpty	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Cuadro 2: Resumen de las complejidades de tiempo para la lista simplemente enlazada, con cola

pushFront Se anticipa $\Theta(1)$ en cualquier caso, porque el flujo del algoritmo es idéntico para cualquier $n > 0$ y la cantidad de trabajo realizada es constante.

pushBack Se anticipa $\Theta(1)$ en cualquier caso por razones idénticas a las dadas para pushFront, gracias al puntero de la cola.

popFront Se anticipa $\Theta(1)$ en cualquier caso por razones idénticas a las dadas para popFront en la implementación anterior.

popBack Se anticipa $\Theta(n)$ en todos los casos, por razones idénticas a las dadas para popBack en la implementación anterior.

find Se anticipa $\Theta(n)$ en el peor caso, $\Theta(1)$ en el mejor caso, y $\Theta(n)$ en el caso promedio, por razones idénticas a las dadas para find en la implementación anterior.

erase Se anticipa $\Theta(n)$ en el peor caso, $\Theta(1)$ en el mejor caso, y $\Theta(n)$ en el caso promedio, por razones idénticas a las dadas para erase en la implementación anterior.

addBefore Se anticipa $\Theta(n)$ en el peor caso, $\Theta(1)$ en el mejor caso, y $\Theta(n)$ en el caso promedio, por razones idénticas a las dadas para addBefore en la implementación anterior.

addAfter Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `addAfter` en la implementación anterior.

isEmpty Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `isEmpty` en la implementación anterior.

2.1.3. Lista doblemente enlazada, sin cola

Operación	Peor caso	Caso promedio	Mejor caso
<code>pushFront</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>pushBack</code>	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
<code>popFront</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>popBack</code>	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
<code>find</code>	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
<code>erase</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>addBefore</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>addAfter</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>isEmpty</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Cuadro 3: Resumen de las complejidades de tiempo para la lista doblemente enlazada, sin cola

pushFront Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `pushFront` en la implementación anterior.

pushBack Se anticipa $\Theta(n)$ en todos los casos, porque siempre se ejecuta una cantidad constante de trabajo por cada elemento de la lista, más una pequeña cantidad de trabajo que es constante e independiente de n .

popFront Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `pushFront`.

popBack Se anticipa $\Theta(n)$ en todos los casos, por razones idénticas a las dadas para `pushBack`.

find Se anticipa $\Theta(n)$ en el peor caso, $\Theta(n)$ en el caso promedio, y $\Theta(1)$ en el mejor caso, por razones idénticas a las dadas para `find` en la implementación anterior.

erase Se anticipa $\Theta(1)$ en todos los casos, porque se realiza siempre una cantidad constante de trabajo gracias al puntero hacia el elemento anterior.

addBefore Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `erase`.

addAfter Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `erase`.

isEmpty Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `isEmpty` en la implementación anterior.

2.1.4. Lista doblemente enlazada, con cola

Operación	Peor caso	Caso promedio	Mejor caso
<code>pushFront</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>pushBack</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>popFront</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>popBack</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>find</code>	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
<code>erase</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>addBefore</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>addAfter</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
<code>isEmpty</code>	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Cuadro 4: Resumen de las complejidades de tiempo para la lista doblemente enlazada, con cola

pushFront Se anticipa $\Theta(1)$ en cualquier caso, porque el flujo del algoritmo es idéntico para cualquier $n > 0$ y la cantidad de trabajo realizada es constante.

pushBack Se anticipa $\Theta(1)$ en cualquier caso por razones idénticas a las dadas para `pushFront`, gracias al puntero de la cola.

popFront Se anticipa $\Theta(1)$ en cualquier caso por razones idénticas a las dadas para `pushFront`.

popBack Se anticipa $\Theta(1)$ en cualquier caso por razones idénticas a las dadas para `pushBack`.

find Se anticipa $\Theta(n)$ en el peor caso, $\Theta(n)$ en el caso promedio, y $\Theta(1)$ en el mejor caso, por razones idénticas a las dadas para `find` en la implementación anterior.

erase Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `erase` en la implementación anterior.

addBefore Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `addBefore` en la implementación anterior.

addAfter Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `addAfter` en la implementación anterior.

isEmpty Se anticipa $\Theta(1)$ en todos los casos, por razones idénticas a las dadas para `isEmpty` en la implementación anterior.

2.2. Cola dinámica

Se implementó una cola usando un arreglo. Cuando el arreglo se llena, los elementos se copian a un arreglo nuevo con el doble de la capacidad. Cuando el arreglo tiene más de tres cuartos de su capacidad libre, los elementos se copian a un arreglo nuevo con la mitad de la capacidad.

2.3. Pila dinámica

La pila se implementó con un arreglo usando la misma estrategia de redimensionamiento que la cola.

3. Mediciones de tiempo

3.1. Listas enlazadas

3.2. Arreglo dinámico

3.3. Cola dinámica

3.4. Comparación entre los métodos análogos de las listas enlazadas y las colas y arreglos dinámicos

4. Conclusiones